CSE755: Generative Machine Learning

Contents

Fundamentals of Generative Models **2**

Autoregressive Models **20**

Fundamentals of Generative Models

Lecture Slides 4–6: Problem Setup, Three Questions, and the Task Picture

Slide 4: What Problem Are We Solving?

The idea. Imagine you have thousands of cat photos. Generative modeling means building a "cat-photo machine" that has learned what cat photos look like in general. It can then do two things:

1. Tell you how believable a photo is. A real cat photo scores high, and a photo of static noise scores very low. This is called **density estimation**.
2. Make brand-new photos that look real but were not in your collection. This is called **sampling**.

The same works for sentences, sounds, molecules or videos. If your data also has labels (like "cat" or "dog"), the machine can learn the image and its label together.

Equation 1: The Dataset

$$D = \{x^{(1)}, \ldots, x^{(N)}\}$$

- **Symbols:** D is your dataset. $x^{(1)}$ is the first example (say, the first photo), and $x^{(N)}$ is the last. N is how many examples you have. The little number in brackets is just a position number, not a power.
- **What it says:** "Here is my pile of N examples."
- **Conditions:** The slides assume the examples are drawn from the same unknown source, independently of each other. Think of each photo as pulled from a giant bag of all possible real photos.
- **Special values:** If N is small, the machine has little to learn from and may just memorize. As N gets very large, the pile represents the real world better, and learning gets more reliable.

Equation 2: The True Rule and the Model's Guess

$$p_{\text{data}}(x) \qquad \text{and} \qquad p_\theta(x)$$

- **Symbols:** $p_{\text{data}}(x)$ is the true, unknown rule for how likely each x is in the real world. $p_\theta(x)$ is our machine's guess at that rule. θ (theta) stands for all the machine's adjustable knobs, such as the weights in a neural network.
- **What it says:** We never see p_{data}. We only see examples from it. The goal is to turn the knobs θ until p_θ behaves like p_{data}.
- **Conditions:** For a list of separate options (like pixel brightness 0–255), p gives a true probability, so it is between 0 and 1. For smooth values, p is a *density*, and a density can be bigger than 1. Only the area under it, over a region, is a probability.
- **Special values:** If $p_\theta(x) = 0$ for something that really exists, the machine is saying "impossible" about something real. That is the worst kind of mistake (the log of 0 is $-\infty$, which training punishes heavily). If $p_\theta(x)$ is high for realistic x and tiny for nonsense, the machine is doing well.

Equation 3: Drawing a New Item

$$x \sim p_\theta$$

- **Symbols:** x is a new item. The wavy "$\sim$" means "is drawn from." p_θ is our model.
- **What it says:** "Roll the machine's dice and get a new item, with more likely items showing up more often."

- **Conditions:** This only works if the model gives you some way to produce samples. Some models do this easily, others are slow.
- **Special values:** Items with p_θ near 0 almost never appear. Items with high p_θ appear often.

Equation 4: Image and Label Together

$$p_\theta(x, y)$$

- **Symbols:** x is the item (image) and y is its label.
- **What it says:** "How likely is this image *together with* this label?"
- **Conditions:** It needs labeled data. If you do not have labels, you drop y and just model $p_\theta(x)$.

Slide 5: Three Questions Every Generative Model Answers

Every generative model tries to answer three questions about data:

Question	Plain meaning	Written as
1. Density	How believable is this x?	$p_\theta(x) = ?$
2. Sample	How do I make a new x?	$x \sim p_\theta$
3. Represent	What hidden factors explain x?	z such that $x \approx g(z)$

Equation 5: Rebuilding From a Hidden Description

$$x \approx g(z)$$

- **Symbols:** x is the real item (like a photo). z is a short hidden description of it, like a recipe card ("fluffy, orange, sitting, facing left"). g is a function that turns the recipe card back into a picture. "$\approx$" means "approximately equal."
- **What it says:** A complicated thing x can be rebuilt, roughly, from a much shorter description z. Finding that z gives a useful summary of x.
- **Conditions:** It is only approximate. A short z cannot hold every tiny detail, so $g(z)$ is close to x but not identical.
- **Special values:** If z is too short, you lose important detail and the rebuilt picture is poor. If z is as big as x itself, nothing is summarized, so it is not very useful.

Which questions each family can answer. The slide's main point is that no model answers all three questions easily. Different model types make different trade-offs:

- **Autoregressive models and flows:** They give an exact answer to question 1 (the exact density). Sampling is exact for flows, and for autoregressive models it happens one piece at a time.
- **VAEs:** They give only a *lower bound* on the density, meaning a number guaranteed not to exceed the true one. In return they give you a hidden code z (question 3).
- **GANs and many diffusion samplers:** They produce very good samples (question 2), but you cannot easily compute the exact $p_\theta(x)$.

"Tractable" just means "can be computed in a reasonable amount of time."

Slide 6: A Picture of the Task

The slide is a simple flow chart:

Dataset $\rightarrow$ (fit θ) $\rightarrow$ **Model** $p_\theta(x) \rightarrow$ two outputs:

- a score for any new item x^*, written $p_\theta(x^*)$, which is its believability (this is the density question)

- a brand-new item x_{new} (this is the sampling question)

Fitting means adjusting the knobs θ until the model's version of the world is *close* to the real one. The slide says "close" will be made precise later using the **Kullback–Leibler divergence** (a score for how different two distributions are) and the **likelihood** (how well the model explains your data).

A new item is written x_{new} (or x^*) to keep it separate from the training examples. The point is that it was not in the dataset.

Summary of slides 4–6: You collect examples, tune θ so the model's distribution matches the real one, and then use the model to score items, create new ones, and (in some model types) find hidden descriptions. Each model family gives up something in exchange for another.

Random Variable, Joint, Marginal, Conditional Probability and Bayes' Rule: General Perspective

One example is used for all five ideas so they connect.

The setup. A school has 100 students. Each one either takes the bus or walks, and each one is either late or on time on a given day. Here is the count for one day:

	Late	**On time**	**Total**
Bus	10	40	50
Walk	5	45	50
Total	15	85	100

Random Variable

A random variable is a way of turning "something that happens by chance" into a number.

If you pick a student at random, you do not know in advance whether they will be late. You can make a rule: "write 1 if the student is late, 0 if on time." That rule is a random variable. It stays unknown until you actually pick someone, and then it becomes a real number.

A dice roll is the classic case. Before you roll, "the number that comes up" is a random variable. After you roll, it is just 4 or 6 or whatever you got.

The reason for using numbers is that you can then count, average and calculate with them.

Joint Probability

Joint probability is the chance of two things being true **at the same time**.

What is the chance a random student takes the bus **and** is late? Look at the one box where both are true: 10 students out of 100, so **10%**.

Every box in the middle of the table is a joint probability:

- bus and late: 10%
- bus and on time: 40%
- walk and late: 5%
- walk and on time: 45%

Together they add up to 100%, because every student lands in exactly one box.

Marginal Probability

Marginal probability is the chance of one thing on its own, ignoring the other.

What is the chance a random student is late, whether they take the bus or walk? Add up the "late" column: $10 + 5 = 15$ out of 100, so **15%**.

You are collapsing the table. You merge the bus and walk rows and look only at the totals. Those totals sit in the *margins* of the table, which is where the name comes from.

Conditional Probability

Conditional probability is the chance of something **once you already know something else is true**.

Suppose I tell you the student takes the bus. What is the chance they are late? You now only care about the bus row, which has 50 students, and 10 of them are late. That gives 10 out of 50, which is **20%**.

For walkers it is 5 out of 50, or **10%**.

You are zooming in on one group and asking what is true inside it. Notice that the answer changed: 15% overall, 20% for bus riders, 10% for walkers. That change tells you that taking the bus and being late are connected. If the numbers had stayed the same for every group, the two things would be unrelated.

You can only zoom into a group that exists. If nobody walked, "chance of being late given they walk" would make no sense, since there is no group to look at.

Bayes' Rule

Bayes' rule is a way to flip a question around.

Here is a question you cannot answer by simply reading off the table: **"A student arrived late. What is the chance they took the bus?"**

Sometimes you do not have the full table. You might only know:

- Half of all students take the bus (50%).
- Bus riders are late 20% of the time.
- Students overall are late 15% of the time.

Bayes' rule says:

$$\text{chance of bus given late} = \frac{\text{chance of late given bus} \times \text{chance of bus}}{\text{chance of late}}$$

With numbers:

$$\frac{0.20 \times 0.50}{0.15} = \frac{0.10}{0.15} \approx 0.67 = 67\%$$

You can check this against the table. Of the 15 late students, 10 took the bus, and $10 \div 15 \approx 67\%$. It matches.

In plain words: Start with what you believed before (half of students ride the bus). Then adjust for the new clue (this student is late), because bus riders are late more often than average. The clue pushes your belief up from 50% to 67%.

A common mix-up: "Chance of late given bus" (20%) and "chance of bus given late" (67%) are *different questions* with different answers. The first asks how many bus riders are late. The second asks how many late students ride the bus. Bayes' rule is the tool for converting one into the other.

How They Fit Together

- A **random variable** turns chance outcomes into numbers.
- **Joint** looks at two things together.
- **Marginal** looks at one thing and ignores the rest.
- **Conditional** looks at one thing inside a group you already know.
- **Bayes' rule** converts "given A, how likely is B" into "given B, how likely is A."

All of them come from the same table, just read in different ways.

Lecture Slides 7–9: Random Variables, Joint, Marginal, Conditional, and Bayes' Rule

Slide 7: Random Variables

The idea. A random variable is a number (or list of numbers) whose value depends on chance. Rolling a die gives a random number from 1 to 6. A pixel's brightness in a random photo is also a random variable. We write X for the random thing itself, x for one specific value it took, and $p(x)$ for the rule saying how likely each value is.

There are two kinds, and each has its own rule.

Equation 1 (Discrete)

$$p(x) \geq 0 \qquad \text{and} \qquad \sum_x p(x) = 1$$

- **Symbols:** x is one possible value (for example, a brightness from 0 to 255). $p(x)$ is the chance of that value. $\sum_x$ means "add up over every possible x."
- **What it says:** Every chance is zero or more (no negative chances), and all the chances together add up to exactly 1, which means 100%. Something must happen.
- **Conditions:** This holds when there is a countable list of separate options, like pixel brightness $0, 1, \ldots, 255$.
- **Special values:** If $p(x) = 0$, that value never happens. If $p(x) = 1$, it always happens, and then every other option must have chance 0. A single $p(x)$ can never be above 1, because the total is only 1.

Equation 2 (Continuous)

$$p(x) \geq 0 \qquad \text{and} \qquad \int p(x)\, dx = 1$$

- **Symbols:** x is a value on a smooth scale, like height or exact temperature. $p(x)$ is now a **density**, not a chance. $\int \ldots dx$ means "add up the area under the curve, across all x."
- **What it says:** The curve never goes below zero, and the total area under it is 1.
- **Conditions:** This holds for smooth, continuous values, where there are infinitely many possible values.
- **Special values:** The slide stresses that $p(x)$ **can be bigger than 1**. Imagine all the area squeezed into a very narrow strip. The strip must be very tall to still have area 1. Only the *area over a region* is a real probability, and that is always between 0 and 1. Also, $p(x)$ at a single exact point is not a probability. The chance of landing on one exact point is 0.

Slide 8: Joint, Marginal, Conditional

The example: x is an image and y is a label, either "cat" or "dog."

Joint

$$p(x, y)$$

- **Symbols:** x is the image, y is the label.
- **What it says:** How likely is this exact image *and* this exact label together?
- **Conditions:** It always makes sense for two variables that live in the same world.
- **Special values:** If it is 0, that image-label pair never occurs (a photo of a cat labeled "dog," say).

Marginal

$$p(x) = \sum_y p(x, y)$$

- **Symbols:** $p(x)$ is the chance of the image on its own. $\sum_y$ means "add up over every label y."
- **What it says:** If you do not care about the label, add up the chances of the image across all labels. This is called "summing the label away."
- **Conditions:** It holds always. For smooth variables, the sum becomes an integral.
- **Special values:** If the image never appears with any label, every term is 0, so $p(x) = 0$.

Conditional

$$p(y \mid x) = \frac{p(x, y)}{p(x)}$$

- **Symbols:** $p(y \mid x)$ reads "chance of label y *given* we already know the image is x." The top is the joint, and the bottom is the chance of the image alone.
- **What it says:** Out of all the times this image shows up, what fraction come with label y? You are zooming in on only the cases where x happened.
- **Conditions:** It only works when $p(x) > 0$. You cannot ask "given that x happened" if x can never happen.
- **Special values:** If $p(x) = 0$, you would divide by zero, so it is undefined. If $p(x, y) = 0$ but $p(x) > 0$, the answer is 0. If the joint equals $p(x)$, the answer is 1, meaning that label is certain.

Product Rule

$$p(x, y) = p(y \mid x)\, p(x) = p(x \mid y)\, p(y)$$

- **Symbols:** Same as above. Both right-hand sides rebuild the joint.
- **What it says:** The chance of "image and label together" equals "chance of the image" times "chance of the label given that image." You can also go the other way: label first, then image given label.
- **Conditions:** It always holds. It is an identity, not an assumption.
- **Special values:** If either piece is 0, the joint is 0.
- **Why it matters:** The two versions give different kinds of models. The first path ($p(y \mid x)\, p(x)$) leads to *discriminative* thinking, which predicts a label from an image. The second ($p(x \mid y)\, p(y)$) leads to *generative* thinking, which picks a label and then creates an image.

Slide 9: Bayes' Rule

The idea. Bayes' rule lets you flip a question around. If you know "how likely is this image if it is a dog," it lets you work out "how likely is it a dog, given this image."

Equation 1: Bayes' Rule

$$p(y \mid x) = \frac{p(x \mid y)\, p(y)}{p(x)}$$

- **Symbols:**
  - $p(y)$, the *prior*: your belief about the label *before* seeing the image. For example, 60% of your photos are cats.
  - $p(x \mid y)$, the *likelihood*: how believable this image is *if* the label were y.
  - $p(x)$, the *evidence*: how likely this image is overall, across all labels.
  - $p(y \mid x)$, the *posterior*: your updated belief *after* seeing the image.
- **What it says:** Start with your old belief, multiply by how well the image fits that label, then divide by a total so everything adds to 1.
- **Conditions:** It needs $p(x) > 0$ (no dividing by zero). It is always true, since it comes straight from the product rule.
- **Special values:**
  - If the prior $p(y) = 0$, the posterior is 0. A label you ruled out stays ruled out, no matter what the image looks like.
  - If the likelihood $p(x \mid y) = 0$, the posterior is 0. The image is impossible under that label.
  - If the prior is the same for all labels, the image evidence decides everything.

Equation 2: Evidence

$$p(x) = \sum_{y} p(x \mid y)\, p(y)$$

- **Symbols:** Add up ($\sum_y$), over every label, the likelihood times the prior.
- **What it says:** The overall chance of an image is built from its chance under each label, weighted by how common each label is. It is the "shrink-to-fit" number that makes the posteriors add to 1.
- **Conditions:** It holds for any complete set of labels (here, cat and dog).
- **Special values:** If it is 0, the image cannot occur and Bayes' rule is undefined.

Quick Example

Say 60% of your photos are cats and 40% are dogs. An image with floppy ears has $p(x \mid \text{cat}) = 0.1$ and $p(x \mid \text{dog}) = 0.5$.

- Cat: $0.1 \times 0.6 = 0.06$
- Dog: $0.5 \times 0.4 = 0.20$
- Evidence: $0.06 + 0.20 = 0.26$
- Posterior: dog $= 0.20/0.26 \approx \mathbf{0.77}$, cat $= 0.06/0.26 \approx \mathbf{0.23}$

So the floppy ears move you from a 40% dog belief to about 77%.

The Diagram: $y \to x$

The arrow shows the *sampling direction*. A generative model picks a label from the prior, then draws an image from $p(x \mid y)$. The posterior $p(y \mid x)$ goes *against* the arrow, and that is the answer a classifier wants.

Summary of slides 7–9:

- Random variables come in two types: chances for separate values and densities for smooth values.
- Joint, marginal and conditional are three ways to look at the same pair of variables.
- Bayes' rule flips a conditional around, which is how a generative model can be used to classify.

Page 10: Independence and the i.i.d. Assumption

This page has three ideas, each with one equation: independence, conditional independence, and i.i.d. data. It sets up the rule that lets us write a whole dataset's probability as a simple product.

Independence

Idea: Two things are independent if knowing one tells you nothing about the other. Example: flipping a coin and rolling a die. The coin result does not change the die's odds.

Equation:

$$p(x, y) = p(x)\, p(y), \qquad \text{equivalently} \qquad p(x \mid y) = p(x)$$

Symbols:

- X and Y are two random quantities (things that can come out different ways).
- x and y are specific values they might take.
- $p(x, y)$ is the chance that $X = x$ and $Y = y$ at the same time.
- $p(x)$ and $p(y)$ are the chances of each one on its own.
- $p(x \mid y)$ is the chance of x once you already know y. The bar "|" means "given".

What it says: The chance of both happening equals the chance of one times the chance of the other. Example: coin heads (1/2) and die shows 6 (1/6) gives 1/12 for both. The second form says the same thing: learning y does not change your odds for x.

When it holds: Only when the two really do not influence each other. It is an assumption you make or a fact you check, not something true by default. The "equivalently" part needs $p(y) > 0$, because you cannot say "given y" about something that never happens.

Special values: If $p(x) = 0$ (x can never happen), then $p(x, y) = 0$ as well, which is consistent. If either probability is 1, the equation just gives the other one back.

Conditional Independence

Idea: Two things can be linked in general, but become unrelated once you know a third thing. Example: ice cream sales and sunburns both rise on hot days, so they look connected. But if you already know the weather (Z), ice cream sales tell you nothing more about sunburns. The weather was the shared cause.

Equation:

$$X \perp Y \mid Z \quad \text{means} \quad p(x, y \mid z) = p(x \mid z)\, p(y \mid z)$$

Symbols:

- $X \perp Y \mid Z$ reads "X is independent of Y, given Z".

- z is the value of the third quantity we already know.
- $p(x, y \mid z)$ is the chance of x and y together, given z.
- $p(x \mid z)$ and $p(y \mid z)$ are the separate chances, each given z.

What it says: It is the same product rule as before, but every term is computed inside the world where z is already known.

When it holds: Only when Z fully explains the link between X and Y. It needs $p(z) > 0$.

Independent Given Z Versus Independent Overall

Independent given Z does **not** mean independent overall (this is mistake number 5 in the hand notes). This matters later because latent variables (hidden causes like Z) are how models explain why pixels in one image depend on each other.

Independent given Z means: if you already know the weather, then ice cream sales and sunburns have nothing more to say about each other.

Independent overall means: even if you know nothing about the weather, knowing one of the two tells you nothing about the other.

The first does not guarantee the second. Picture two kinds of days.

- **Hot days:** lots of ice cream sold, lots of sunburns.
- **Cold days:** little ice cream sold, few sunburns.

Once you tell me "it is a hot day", ice cream sales do not help me predict sunburns any better. The heat already explains both. The same is true on a cold day. So **given the weather, they are unrelated.**

Now forget the weather. I tell you, "Ice cream sales were huge today." You would guess it was probably hot, so you would also guess there were lots of sunburns. Knowing one helped you predict the other. So **overall, they are connected.**

"Ignoring the weather" means you pool all the days together, hot and cold. Hot days bring high sales and high sunburns, and cold days bring low sales and low sunburns. So across all days, high sales and high sunburns tend to show up together. That is the connection.

Think of Z as a hidden middleman. It makes the two things move together, so they look related from outside. Once you look inside one weather type, the link disappears.

Checking against the definitions. The definition of "independent overall" says that, with no weather information, knowing one thing tells you nothing about the other. The ice cream example fails that test, so ice cream sales and sunburns are **not** independent overall.

Situation	Does knowing one help guess the other?	Verdict
Coin and die	No	Independent overall
Ice cream and sunburns, weather unknown	Yes	**Not** independent overall
Ice cream and sunburns, weather known	No	Independent **given the weather**

The bottom two rows are the point of the lesson. The same pair can be connected when you do not know Z and unconnected once you do.

Why this can sound contradictory. The sentence "independent overall means no connection" is the rule, and the later sentence "overall, they are connected" says the example breaks the rule. Both are true.

Why it matters for images. In a photo, neighboring pixels are related. If one pixel is blue sky,

the one beside it is probably blue too. A generative model explains this using a hidden cause, like the weather in the example. Maybe the hidden cause is "this photo is of a beach at noon". Given that, each pixel is easy to describe on its own. But since all the pixels share the same hidden cause, they end up connected when you do not know the cause.

This hidden cause is called a **latent variable** (Z). "Latent" just means hidden. The model never sees Z in the training data. It invents one and uses it to explain why pixels move together.

In one sentence: Things can look connected only because they share a hidden cause. Once you know the cause, the connection disappears, and generative models use this trick to explain why the pixels in an image depend on each other.

i.i.d. Data

Idea: i.i.d. means "independent and identically distributed". It describes how we assume the training data was collected.

Equation:

$$p(\mathcal{D} \mid \theta) = \prod_{n=1}^{N} p\big(x^{(n)} \mid \theta\big)$$

Symbols:

- $\mathcal{D}$ is the whole dataset.
- θ (theta) is the model's settings or knobs, the numbers we tune.
- N is how many data points there are.
- $x^{(n)}$ is the n-th data point. The small raised (n) is a label saying which point, not a power.
- $p(\mathcal{D} \mid \theta)$ is the chance of seeing this entire dataset, given the settings θ.
- The big $\prod$ means "multiply all of these together", from point 1 up to point N.

What it says: The chance of the whole dataset is the chance of point 1, times the chance of point 2, and so on. Two assumptions are packed in:

- **Identical:** every point comes from the same model with the same θ. One set of knobs for everyone.
- **Independent:** the points do not affect each other, so multiplying is allowed. This is the same product rule from independence, applied N times.

When it holds: When each data point is collected separately and fairly, like photos shuffled from one big collection. It fails if points are linked, such as consecutive frames of a video, or if data comes from different sources with different θ.

Special values:

- $N = 1$: the product is just one term.
- Any single factor is 0: the whole product becomes 0. One impossible point makes the entire dataset impossible under that θ.
- N very large: you multiply many numbers smaller than 1, so the result shrinks toward 0 and gets too tiny for a computer to store (underflow). This is exactly why page 11 switches to logarithms, which turn the product into a sum.

The last sentence of the page. "Pixels inside one image are not independent." Independence is assumed *between* different images (point n versus point m), not *inside* one image. Neighboring pixels in a photo are strongly related: if one is blue sky, its neighbor probably is too. If pixels were independent, an image would look like random static. Capturing that pixel-to-pixel dependence is the real challenge,

and that is the job of a generative model.

Likelihood and Maximum Likelihood Estimation: General Perspective

Probability Versus Likelihood

Imagine a coin with an unknown chance of heads, called θ.

- **Probability** starts with the rule and asks about results. "The coin is fair ($\theta = 0.5$). How likely is it to get 7 heads in 10 flips?" The rule is known and the results are the question.
- **Likelihood** starts with the results and asks about the rule. "I flipped 10 times and got 7 heads. How believable is $\theta = 0.5$? What about $\theta = 0.7$?" The results are known and the rule is the question.

Both use the same formula, but you look at it from opposite sides.

In one sentence: Probability asks, "Given the rules, what will I see?" Likelihood asks, "Given what I saw, how well does this set of rules fit?"

A Worked Example

Say you saw one specific sequence with 7 heads and 3 tails. Its chance under a coin with heads-chance θ is

$$\theta^7(1-\theta)^3,$$

and that is the likelihood of θ.

- $\theta = 0.5$ gives $0.5^{10} \approx 0.00098$.
- $\theta = 0.7$ gives $0.7^7 \times 0.3^3 \approx 0.00222$.

$\theta = 0.7$ explains the data about 2.3 times better, so it is the more believable setting. Likelihood does not behave like probability across θ. If you add up the likelihood over all possible θ values, you do not get 1. It is only a score for comparing settings.

Maximum Likelihood Estimation (MLE)

MLE says: try every setting θ and pick the one that makes the data we saw most likely. Here that is $\theta = 7/10 = 0.7$. It is a way of letting the data choose the settings.

Why We Use Logarithms

With 1,000 data points, you multiply 1,000 small numbers. If each is about 0.1, the result is 10^{-1000}, which is too tiny for a computer to store, so it rounds to 0. Taking the log turns multiplication into addition, and the numbers stay manageable (about -2303). Because log never reverses order (bigger stays bigger), the best θ stays the same.

Page 11: Likelihood Is a Function of the Parameters

The Two Boxes

- **Probability box:** θ is fixed and x (the data) varies. $p(x \mid \theta)$ is a distribution over possible data.
- **Likelihood box:** the data is fixed and θ varies. $L(\theta) = p(\mathcal{D} \mid \theta)$.

Symbols:

- x is one possible data point (for us, maybe an image).
- θ is the model's settings, the knobs we tune.

- $\mathcal{D}$ is the full training dataset.
- $p(x \mid \theta)$ is the chance of x, given the knob settings θ.
- $L(\theta)$ is the likelihood, a score for each knob setting.

What it says: Same formula, different viewpoint. With θ fixed, adding up $p(x \mid \theta)$ over all possible x gives 1. With the data fixed, $L(\theta)$ is just a score and does not add up to 1. It only becomes a real probability over θ if you add a prior, meaning a starting belief about θ before seeing data (that is the Bayes' rule setup from page 9).

Special values: If $L(\theta) = 0$, those settings make your data impossible. In the coin example, $\theta = 1$ (always heads) gives likelihood 0 if you saw even one tail.

Probability Versus Likelihood from a Generative Model's Perspective

The setup. A generative model is an **image-making machine** with knobs. The knobs are θ (in real models, millions of numbers inside a neural network). Turn the knobs and the machine makes different kinds of images.

Toy machine: it draws a strip of 4 pixels, and each pixel comes out white with chance θ, otherwise black. The only knob is θ.

Probability: knobs fixed, images vary. "The knob is set to $\theta = 0.75$. How likely is it that the machine makes this image?"

- White-white-white-black: $0.75 \times 0.75 \times 0.75 \times 0.25 \approx 0.105$
- All four white: $0.75^4 \approx 0.316$

Here the machine is fixed and you ask about the images it might produce. If you add up the chances of all 16 possible strips, you get exactly 1, because the machine must make something. Probability is about **what the machine will make**.

Likelihood: images fixed, knobs vary. Now flip it. You have real training images and do not know the right knob setting. Say your training set has 2 images: WWWB and WWBW (6 white pixels out of 8).

You ask: "For each possible knob setting, how likely is it that the machine would have made **these exact images**?" The formula is $\theta^6(1-\theta)^2$.

Knob setting θ	Score for the training images
0.25	0.00014
0.50	0.0039
0.75	**0.0111**

Here the images are fixed and you compare knob settings. Adding the scores across all possible θ does **not** give 1. They are only scores for comparing settings. Likelihood is about **how well a knob setting explains the data you have**.

How the two connect. It is the same formula, $p(\text{images} \mid \theta)$, read in two ways:

- **Probability:** hold θ fixed, vary the images.
- **Likelihood:** hold the images fixed, vary θ.

Where training comes in. Training a generative model means: take the real training images (fixed) and keep adjusting the knobs until the likelihood is as high as possible. In the toy machine that lands on $\theta = 0.75$, which is just 6 white pixels out of 8.

A trained model then switches to the probability view. With knobs frozen, it can **generate** new images (what will the machine make?) or **score** an image (how likely is this one?).

One line: Likelihood is how you *train* the machine (fix the data, adjust the knobs). Probability is how you *use* it (fix the knobs, produce or score images).

The i.i.d. Likelihood and Log-Likelihood Equations

Equations:

$$L(\theta) = \prod_{n=1}^{N} p\big(x^{(n)} \mid \theta\big),$$

$$\ell(\theta) = \log L(\theta) = \sum_{n=1}^{N} \log p\big(x^{(n)} \mid \theta\big)$$

Symbols:

- N is the number of data points, and $x^{(n)}$ is the n-th one.
- $\prod$ means multiply all the terms, and $\sum$ means add all the terms.
- ℓ (curly "l") is the log-likelihood.
- log is the natural logarithm.

What it says: The likelihood of the whole dataset is the product of each point's chance. Taking the log changes the product into a sum.

When it holds: The product form needs the i.i.d. assumption from page 10: independent points and one shared θ. Without it you cannot split the dataset into separate multiplied pieces.

Special values:

- If any single point has $p = 0$, the product is 0, and its log is $-\infty$. One impossible point ruins the whole score.
- For discrete data (like pixel values 0 to 255), each p is between 0 and 1, so each log is zero or negative. A p of 1 gives $\log = 0$, and p near 0 gives a very large negative number.
- For continuous data, p is a density and can exceed 1 (page 7), so a log term can be positive.
- As N grows, the product shrinks toward 0 and ℓ becomes a larger and larger negative number. That is why people often divide by N.

Slide's reason: "Products of probabilities underflow, and log is strictly increasing." Underflow means the number gets too tiny to store. Strictly increasing means a bigger L always gives a bigger $\log L$, so the same θ wins.

Generative angle: Here $p(x \mid \theta)$ is the generative model. The likelihood answers: "If my model were the true source of images, how surprised would it be by the training set?" High likelihood means the training images look typical to the model.

Page 12: Maximum Likelihood Estimation

The MLE Equation

Equation:

$$\hat{\theta}_{\mathrm{ML}} = \arg\max_{\theta} \ell(\theta) = \arg\max_{\theta} \sum_{n=1}^{N} \log p\big(x^{(n)} \mid \theta\big)$$

Symbols:

- $\hat{\theta}$ (theta-hat) is our best-guess settings. The hat means "estimated from data".

- $\arg\max_\theta$ means "the θ that makes what follows biggest". It returns the setting, not the score.
- The rest is the log-likelihood from page 11.

What it says: Try all knob settings and keep the one that gives the training data the highest score.

When it holds / caveats:

- The best setting must exist. For simple models there is one clear peak.
- For deep neural networks there are many peaks, so training (gradient descent) usually finds a good one, not necessarily the best one.
- With very little data, MLE can overfit, locking onto quirks of those few points.

Special values: For the coin, MLE gives N_1/N. If you saw 0 heads in 5 flips, $\hat{\theta} = 0$, so the model declares heads impossible. That is a known weakness.

Generative angle: Training a generative model means adjusting its knobs (for example, a neural network's weights) so the real training images become as likely as possible under the model.

Why This Objective: The KL Equation

Equation:

$$\mathbb{E}_{p_{\text{data}}}\big[\log p_\theta(x)\big] = -H(p_{\text{data}}) - \mathrm{KL}\big(p_{\text{data}} \,\|\, p_\theta\big)$$

Symbols:

- p_{data} is the true, unknown process that made the real data.
- p_θ is our model's version.
- $\mathbb{E}_{p_{\text{data}}}$ means "the average over data drawn from the real world".
- $\log p_\theta(x)$ is the log-score our model gives to a real example x.
- $H(p_{\text{data}})$ is entropy, a measure of how unpredictable the real data is. It depends only on the real world, not on our model.
- $\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$ is the Kullback–Leibler divergence, a score for how different our model is from reality. It is zero only if they match exactly and positive otherwise.

What it says: The average score our model earns on real data equals a fixed number minus the gap between model and reality. Since the fixed number cannot be changed, raising the score means shrinking the gap. So maximum likelihood is the same as making the model as close to reality as possible, measured by KL.

Link to the sum: We cannot average over the true world, but the average over our N training points,

$$\frac{1}{N}\sum_{n=1}^{N} \log p_\theta\big(x^{(n)}\big),$$

is a stand-in for it. That connects page 11's sum to this equation.

When it holds: It is an exact identity when both are valid distributions. KL is finite only if the model gives nonzero chance everywhere real data can occur.

Special values:

- $\mathrm{KL} = 0$: perfect match.
- Model chance $\to 0$ on something real data does produce: $\log \to -\infty$, so $\mathrm{KL} \to \infty$. The model is punished severely for calling real things impossible.
- A low-entropy (very predictable) world means a higher best-possible score.

Direction of KL Matters

KL is not symmetric. $\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$ and $\mathrm{KL}(p_\theta \,\|\, p_{\text{data}})$ are different scores.

Imagine real data has two clusters: cat photos and dog photos.

- $\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$ (**what MLE minimizes**): The model is punished heavily if it gives almost no chance to something real. Missing the dogs is very costly, so the model spreads out to cover **both** clusters. This is "mode covering" (a mode is a peak or cluster). The downside is that spreading wide can put some mass in between, on things that look like neither, which can cause blurry or odd samples.
- **Reverse** $\mathrm{KL}(p_\theta \,\|\, p_{\text{data}})$: The model is punished for putting chance where no real data exists. It can safely focus on just the cats and ignore the dogs. This is "mode seeking". Its samples tend to look cleaner but miss variety.

Generative angle: This explains the trade-offs among model families later. Likelihood-trained models tend to cover all the variety, and some variational methods lean toward the reverse behavior.

Worked Example: Bernoulli MLE

The setup. You flip a coin N times and record 1 for heads and 0 for tails. You do not know the coin's bias, so you give it a name, θ (the chance of heads). The question: which θ best explains the flips you saw?

Step 1: score each θ. If you saw N_1 heads and N_0 tails, the chance of that exact sequence is θ multiplied N_1 times, times $(1-\theta)$ multiplied N_0 times. Taking the log turns this product into a sum:

$$\ell(\theta) = N_1 \log \theta + N_0 \log(1-\theta)$$

This score is high when θ makes your data look likely and low when it does not.

Step 2: find the θ with the highest score. Calculus finds the peak: the slope of the score is zero at the top of the hill. Setting the slope to zero gives

$$\frac{N_1}{\theta} = \frac{N_0}{1-\theta},$$

which simplifies to

$$\hat{\theta} = \frac{N_1}{N}.$$

Step 3: confirm it is a peak, not a valley. The second derivative is negative everywhere between 0 and 1. That means the curve bends downward, so the point you found is a maximum.

What it means. The best guess for the coin's bias is the fraction of heads you saw. With 7 heads in 10 flips, $\hat{\theta} = 0.7$. You can check that it beats alternatives: $0.7^7 \times 0.3^3 \approx 0.0022$, while a fair coin gives $0.5^{10} \approx 0.001$.

Why the notes include it. Here the best answer comes out as a one-line formula. In deep generative models, the goal is the same (pick the parameters that make the training data most likely), but $\log p_\theta(x)$ is too complicated for a formula. You have to improve it step by step with gradient descent, replace it with an easier-to-compute lower bound, or drop the idea entirely (GANs).

What a Density Is Good For

Once your model gives a number $p_\theta(x)$ for how likely any x is, you can use it for four things.

1. Compression. Likely things can be written with short codes, and unlikely things need long ones. An item with probability p needs about $-\log_2 p$ bits. Image papers report *bits per dimension*: the total bits for an image divided by the number of values in it (height $\times$ width $\times$ 3 for an RGB image).

A model that knows nothing treats all 256 values per pixel channel as equally likely, which costs 8 bits each. A useful model must beat 8, because it has learned that images have structure (neighboring pixels are similar, for instance). Lower is better.

2. Filling in missing parts. If the model understands how all the pieces of x relate, you can fix some pieces (the pixels you can see) and ask what the rest probably looks like. That is how you patch a hole in an image or guess a missing word.

3. Spotting oddballs. If an input gets a very low $p_\theta(x)$, it is probably unusual, so you can flag it. The notes warn that this is unreliable for high-dimensional data like images: some strange inputs still get high scores. Treat it as a hint, not proof.

4. Classifying with Bayes' rule. If you have a model of what each class looks like, $p(x \mid y)$, and how common each class is, $p(y)$, you can ask which class most likely produced a new input. That gives you a classifier from a generative model.

Likely and Unlikely Things in Compression

By "likely" we mean things the model gives a high probability, and "unlikely" means things it gives a low probability.

The idea behind it. Think of Morse code. The letter "E" is very common in English, so it gets a very short code (a single dot). The letter "Q" is rare, so it gets a longer one. Overall, messages get shorter because the frequent stuff is cheap and only the rare stuff is expensive.

Compression with a probability model works the same way. For an item x that the model gives probability $p(x)$, the best possible code length is about $-\log_2 p(x)$ bits:

- $p(x) = 1/2$ gives 1 bit
- $p(x) = 1/8$ gives 3 bits
- $p(x) = 1/1024$ gives 10 bits

So higher probability means a shorter code, and lower probability means a longer one.

In the image setting. The "things" are whole images (or individual pixel values). A model that has learned what images typically look like gives high probability to realistic ones, such as smooth sky or a plausible face, and very low probability to random static. Realistic images then get short codes, and noise gets a long code.

Why the 8-bit baseline matters. A model that knows nothing treats every pixel value from 0 to 255 as equally likely, so each has probability $1/256$, which costs $\log_2 256 = 8$ bits. A good model does better because it has learned that some values are much more probable than others given the surrounding pixels. That is why a lower bits-per-dimension number means a better density model.

A note on precision. For continuous data, the density value itself is not a probability, so in practice the pixels are treated as discrete (0 to 255) when computing bits per dimension.

Generative and Discriminative Models

The starting point: one joint, two ways to split it

Take an image x and a label y (cat or dog). The product rule says the joint probability can be split in two equally valid ways:

$$p(x, y) = p(y \mid x)\, p(x) = p(x \mid y)\, p(y)$$

Both are true. The difference is which pieces you choose to learn.

Discriminative model

It learns only $p(y \mid x)$: given an image, how likely is each label? Logistic regression and a ResNet classifier are examples.

The model's whole job is to answer "which label?", so it never learns how images are produced. That means it **cannot draw a new image**. It can look at an image and name it, but not create one.

Generative model

It learns $p(x)$, or $p(x \mid y)$ together with $p(y)$, or the full joint $p(x, y)$. PixelCNN, language models, VAEs and diffusion models are examples.

Because it models how the data itself looks, you can draw new samples from it. The notes say that sampling is part of the definition, even if it is slow or inconvenient.

The arrows diagram

The arrows show the direction in which data is *produced*:

- **Discriminative:** image → label. It reads an image and outputs a label.
- **Generative:** label → image. It picks a label, then generates an image that fits it. An unconditional generative model skips the label and just draws an image directly.

The comparison table

	Discriminative	Generative
What it learns	$p(y \mid x)$	$p(x)$ or $p(x, y)$
Can create new x?	No	Yes
Missing pixels	Needs a separate model	Fill them in using the joint
Difficulty of the job	Only has to explain the label	Has to explain all of x
Typical failure	Confidently wrong on inputs unlike its training data	Blurry output, missing some kinds of data, slow sampling

The "difficulty" row is the key one. Predicting a label is a small job. Explaining every pixel of an image is a huge one.

The Ng and Jordan (2001) result

This was a comparison on classification only:

- With **lots of data**, a discriminative classifier eventually reaches a **lower error**.
- With **little data**, a generative classifier often gets to *its* best performance sooner, because it also learns the structure of what each class looks like. But its best performance can be worse.

So the notes make the point that being generative is not about classifying better. You choose it when you want **samples, densities, or a learned representation**.

Generative classification: the classic recipe

A generative model can still classify. Learn $p(y)$ (how common each class is) and $p(x \mid y)$ (what each class looks like), then pick:

$$\hat{y} = \arg\max_{y} \; p(x \mid y)\, p(y)$$

In words: for a new input, ask for each class "how well does this look like that class, and how common is that class?", then pick the class with the highest score. (This is Bayes' rule with the denominator dropped, since $p(x)$ is the same for every class and does not change which one wins.)

If each class is modeled as a bell-shaped cloud (a Gaussian):

- **Same cloud shape for all classes** gives straight-line decision boundaries. This is **LDA**.
- **Each class gets its own shape** gives curved boundaries. This is **QDA**.

Deep generative models keep this same idea but make two swaps:

1. Replace the simple Gaussian with a neural network that can model far more complex shapes.
2. Replace the discrete label y with a continuous hidden variable z.

Autoregressive Models

Chain Rule of Probability: How and Why It Works

1. Where it comes from: conditional probability

Everything starts from the definition of conditional probability:

$$p(A \mid B) = \frac{p(A, B)}{p(B)}, \quad \text{for } p(B) > 0$$

Read it as: "Among all outcomes where B happened, what fraction also has A?"

Multiply both sides by $p(B)$:

$$p(A, B) = p(B)\, p(A \mid B)$$

This is the **two-variable chain rule**. It is just the definition rearranged, not a new law.

2. Extending to many variables

Apply the same step repeatedly, treating everything before the current variable as "B".

For three variables, group (x_1, x_2) as one block:

$$p(x_1, x_2, x_3) = p(x_1, x_2)\, p(x_3 \mid x_1, x_2)$$

Then split $p(x_1, x_2)$ again:

$$p(x_1, x_2, x_3) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2)$$

Repeating this D times gives the general form:

$$p(x_1, \ldots, x_D) = \prod_{i=1}^{D} p(x_i \mid x_{<i})$$

3. Why it works: the product telescopes

Write each conditional using its definition:

$$p(x_1) \cdot \frac{p(x_1, x_2)}{p(x_1)} \cdot \frac{p(x_1, x_2, x_3)}{p(x_1, x_2)}$$

Each numerator cancels the next denominator. Only $p(x_1, x_2, x_3)$ survives.

So the chain rule is an **identity**. It holds for every distribution and makes no assumptions:

- **No independence is assumed.** Each variable may depend on all earlier ones.
- **Nothing is approximated.** The product equals the joint exactly.

The only requirement is that each prefix has nonzero probability, so the divisions are defined.

4. Intuition: narrowing down step by step

Think of the joint probability as a fraction of all possible outcomes. You reach it by successive filtering.

Take a language model generating the sentence **"the cat sat"**:

Step	Factor	Meaning	Value
1	$p(\text{the})$	Fraction of all sentences that start with "the"	0.05
2	$p(\text{cat} \mid \text{the})$	Of those, fraction whose next token is "cat"	0.01
3	$p(\text{sat} \mid \text{the, cat})$	Of those, fraction whose next token is "sat"	0.20

$$p(\text{the cat sat}) = 0.05 \times 0.01 \times 0.20 = 0.0001$$

Each factor is a "fraction of a fraction." Multiplying them gives the share of the whole sentence space occupied by exactly this sequence.

The same logic holds for images. The probability of a whole image equals:

- the probability of the first pixel's intensity,
- times the probability of the second pixel given the first,
- and so on, up to the last pixel given all pixels before it.

5. Contrast: what would be wrong

If you assumed the tokens were independent, you would compute:

$$p(\text{the})\, p(\text{cat})\, p(\text{sat})$$

This ignores the fact that "sat" is far more likely after "the cat" than in general. The joint would be badly wrong.

The chain rule keeps every dependency by conditioning on the full history.

6. Order does not matter for correctness

The derivation never used a particular order. You could factor right-to-left, or in any permutation:

$$p(x_1, x_2, x_3) = p(x_3)\, p(x_2 \mid x_3)\, p(x_1 \mid x_2, x_3)$$

All orderings give the **same joint**. What changes is the set of conditionals. Some orderings are easier for a network to learn, such as left-to-right for text, time order for audio, and raster order for images. That is why the lecture says "the order is part of the model."

7. Bridge to autoregressive models

The chain rule turns one huge, intractable object into D small conditionals. Examples of that huge object are a distribution over all $32 \times 32 \times 3$ images, or over all possible sentences.

An autoregressive model's only job is to learn those conditionals with a shared network. Because the factorization is exact, the model's likelihood is exact too.

Meaning of "The Chain Rule Is an Identity"

An **identity** is an equation that is true for every possible value of its variables, not just under certain conditions. For example, $(a+b)^2 = a^2 + 2ab + b^2$ holds for all numbers a, b; you never have to check whether it applies.

"The chain rule is an identity" means:

$$p(x_1, \ldots, x_D) = \prod_{i=1}^{D} p(x_i \mid x_{<i})$$

is true for **every** probability distribution and **every** sequence. It follows directly from the definition of conditional probability, as the telescoping cancellation showed, so it cannot fail.

Contrast this with a **modeling assumption**, which can be wrong:

- **Identity (always true):** $p(\text{the cat sat}) = p(\text{the})\, p(\text{cat} \mid \text{the})\, p(\text{sat} \mid \text{the, cat})$

- **Assumption (may be false):** $p(\text{the cat sat}) = p(\text{the})\,p(\text{cat})\,p(\text{sat})$, which claims the tokens are independent

Why it matters for autoregressive models: writing the joint as a product of conditionals loses no generality. Any distribution over images, text, or audio can be represented this way.

The only possible source of error is the network's estimate of each conditional $p_\theta(x_i \mid x_{<i})$, not the factorization itself. That is why the lecture calls the likelihood "exact": there is no approximation gap like the ELBO in VAEs.

Chain Rule for Dependent vs. Independent Variables

The chain rule does **not** assume dependence. It *allows* dependence but does not *require* it. It holds in both cases.

What happens when variables are independent

Independence means that knowing the past tells you nothing about the next variable:

$$p(x_i \mid x_{<i}) = p(x_i)$$

Substitute this into the chain rule:

$$p(x_1, \ldots, x_D) = \prod_i p(x_i \mid x_{<i}) = \prod_i p(x_i)$$

This is exactly the familiar product rule for independent variables. So the independence formula is just a **special case** of the chain rule, where each conditional simplifies to a marginal.

Generative example

Suppose a model generates **white-noise audio**, where each sample is drawn independently from $\mathcal{N}(0, \sigma^2)$:

- The chain rule still gives $p(x_1)\,p(x_2 \mid x_1)\,p(x_3 \mid x_1, x_2)\cdots$
- Because the samples are independent, each $p(x_i \mid x_{<i})$ equals $\mathcal{N}(0, \sigma^2)$.
- The result is a product of identical Gaussians, which is correct.

Now suppose it generates **speech**, where each sample depends strongly on the previous ones:

- The conditionals carry that dependence.
- The chain rule still holds, and now the conditioning actually matters.

The one real condition

The only requirement is that each prefix must have nonzero probability, so $p(x_i \mid x_{<i})$ is defined.

This condition is about the conditional being well-defined. It has nothing to do with dependence or independence. For a valid sequence you are evaluating, it is satisfied automatically.

Summary

Situation	Chain rule holds?	What the conditionals look like
Dependent (text, speech, images)	Yes	Depend on the history
Independent (white noise)	Yes	Reduce to plain marginals

This is why it is called an identity: it is true regardless of the dependence structure. Dependence only changes what the conditionals turn out to be.

Definition of an Autoregressive Model

From the lecture: *"An autoregressive (AR) model represents the joint by the chain-rule product and parameterizes each conditional with a shared function of the past."*

The sentence, broken into parts

1. "represents the joint by the chain-rule product"

The model's goal is the joint distribution $p(\mathbf{x})$, for example a distribution over all possible images or sentences. Instead of modeling it directly, the model writes it as

$$p(\mathbf{x}) = \prod_{i=1}^{D} p(x_i \mid x_{<i})$$

Since the chain rule is an identity, this loses nothing. The problem has changed from "model the whole object at once" to "model D next-step predictions."

2. "parameterizes each conditional"

Each factor $p(x_i \mid x_{<i})$ is unknown, so a neural network with parameters θ is used to produce it. "Parameterize" means the conditional's form is set by learnable numbers θ, written $p_\theta(x_i \mid x_{<i})$.

3. "with a shared function"

One network with **one** set of parameters θ produces every conditional. You do not train a separate network for position 1, position 2, and so on.

Sharing is necessary, not just convenient. Take a $32 \times 32 \times 3$ image with 256 intensity values:

- Without sharing, the conditional for position i would need an entry for every possible prefix, which is 256^{i-1} cases.
- At position 3072, that number is astronomically large, so no lookup table could store it.

A shared network avoids this. It learns general rules, such as "pixels near an edge tend to continue that edge," and applies them at every position.

4. "of the past"

The network may look **only** at $x_1, \ldots, x_{i-1}$. It must never see x_i itself or anything after it. This is the causal constraint. If the network saw x_i, it could simply copy the answer, and the product would no longer equal the true joint.

The equation

$$p_\theta(x_i \mid x_{<i}) = f_\theta(x_1, \ldots, x_{i-1})$$

Symbol	Meaning
f_θ	The neural network: an RNN, masked CNN, or Transformer
θ	Its parameters, the same at every position i
Input $x_1, \ldots, x_{i-1}$	The past only, which is the context
Output	A **probability distribution** over the possible values of x_i

The key point is that f_θ outputs a **distribution**, not a single predicted value.

Example: generating text. Suppose the context is "the cat" and the vocabulary has 50,000 tokens. Then f_θ(the, cat) outputs a 50,000-entry probability vector via softmax:

- "sat" $\rightarrow$ 0.20
- "is" $\rightarrow$ 0.15

- "ran" $\to 0.08$
- ... and so on, summing to 1.

Uses of this vector:

- **Training:** read off the probability of the true next token and add its $-\log$ to the loss.
- **Generation:** sample a token from this vector.

Example: generating an image pixel. The context is all pixels above, plus those to the left in the current row. f_θ outputs a 256-way softmax over the intensities of the next channel value.

For an audio sample, it could instead output the parameters of a Gaussian or a mixture of logistics.

The "auto" part

"Autoregressive" means the variable is predicted (*regressed*) from earlier values of **itself** (*auto*). Pixels predict pixels, tokens predict tokens, audio samples predict audio samples. There is no separate input stream.

This differs from, say, a translation model, which predicts French from an English input. The conditional PixelCNN in the lecture adds an external $\mathbf{h}$, but the autoregressive core is still conditioning on the data's own past.

The Chain Rule Holds for Every Permutation of the Coordinates

What the statement means

The chain rule is the definition of conditional probability applied repeatedly:

$$p(a, b) = p(a)\, p(b \mid a) = p(b)\, p(a \mid b).$$

Nothing in this identity says *which* variable goes first. So with D variables you can choose any ordering π (a permutation of $1, \ldots, D$), reveal the variables in that order, and the product of conditionals still equals the same joint:

$$p(\mathbf{x}) = \prod_{i=1}^{D} p\big(x_{\pi(i)} \mid x_{\pi(1)}, \ldots, x_{\pi(i-1)}\big).$$

Here $\pi(i)$ means "the coordinate visited at step i."

Text example

Take the 3-token sentence "the cat sat", with $x_1 =$ the, $x_2 =$ cat, $x_3 =$ sat.

Left-to-right, $\pi = (1, 2, 3)$, which is how GPT works:

$$p(\text{the}) \cdot p(\text{cat} \mid \text{the}) \cdot p(\text{sat} \mid \text{the, cat})$$

Right-to-left, $\pi = (3, 2, 1)$:

$$p(\text{sat}) \cdot p(\text{cat} \mid \text{sat}) \cdot p(\text{the} \mid \text{cat, sat})$$

Middle-first, $\pi = (2, 1, 3)$:

$$p(\text{cat}) \cdot p(\text{the} \mid \text{cat}) \cdot p(\text{sat} \mid \text{the, cat})$$

All three products give exactly the same number, $p(\text{the, cat, sat})$. The individual factors are different questions, though:

- $p(\text{cat} \mid \text{the})$ asks which noun follows "the".
- $p(\text{cat} \mid \text{sat})$ asks what kind of subject sits.

Image example

For a 32×32 image, raster order predicts the bottom-right pixel last, using the whole image above it. If you reverse the order, the bottom-right pixel comes *first* and has no context, so its conditional is just the marginal distribution of corner pixel colors. A column-major order or a space-filling curve gives yet another set of conditionals. The image's joint probability is identical under all of them.

Why it matters: the order is part of the model

1. **In theory, the joint is the same.** If the network learned every conditional perfectly, every order would give the same $p(\mathbf{x})$.
2. **In practice, the learning problem changes.** The network f_θ has to approximate a different set of functions for each order. Some orders are easier: text left-to-right matches how grammar unfolds, and audio forward in time matches causality. For images, no order is naturally correct; raster order is a convention.
3. **Learned models are not order-invariant.** Two trained models with different orders usually assign *different* probabilities to the same image, because each one's approximation errors fall in different places. Whichever order makes the conditionals easier to learn tends to get better bpd.
4. **Sampling follows the order.** Ancestral sampling must generate in order π. That decides which parts of the output are produced with little context (and are most uncertain), and where exposure-bias errors start to compound.

In one line: the chain rule guarantees that every order is *exactly correct* as a factorization, but you still have to pick one. That choice determines which conditionals the network must learn and the order in which samples are generated.

Each Order Defines a Different Set of Functions for f_θ

Each conditional is a function

In an AR model, every factor $p(x_i \mid x_{<i})$ is a **function**. Its input is the context (the variables already revealed) and its output is a probability distribution over the next variable. The network f_θ, with one set of shared weights θ, has to compute all D of these functions.

The order π fixes two things for each step:

- **which variables are the input** (what the network can see), and
- **which variable is the target** (what it must predict).

Change the order and both change, so the network is being asked to compute different input→output mappings.

Text example

The functions the network must implement for "the cat sat" under each order:

Left-to-right, $\pi = (1, 2, 3)$:

Step	Input	Output: distribution over ...
1	nothing	first word of a sentence
2	"the"	word after "the"
3	"the cat"	word after "the cat"

Right-to-left, $\pi = (3, 2, 1)$:

Step	Input	Output: distribution over ...
1	nothing	last word of a sentence
2	"sat"	word before "sat"
3	"cat sat"	word before "cat sat"

These are different jobs:

- Step 1 in left-to-right learns "sentences often start with *the, I, a*". In right-to-left, step 1 must learn "sentences often end with *it, here, sat*".
- Step 2 in left-to-right learns that a noun usually follows a determiner. In right-to-left, it learns which subjects tend to precede a given verb.

The data is the same and so is the joint $p(\text{the, cat, sat})$, but the training targets and their inputs are completely different. The weights θ end up encoding different knowledge.

Image example

Predicting pixel $(16, 16)$ in a 32×32 face image:

- **Raster order:** the input is the top half plus the left part of row 16. The function maps "forehead, eyes, part of the nose" to a distribution over nose-pixel colors. That is fairly easy, because the face structure is already established.
- **Reverse raster:** the input is the bottom half (mouth, chin). The function now maps "mouth and chin" to nose color. That is a different function.
- **Center-first order:** this pixel might come first, with no input at all. The function is then just "what color is the center of any image", which is very uncertain.

Why this matters

1. **Difficulty differs.** Some functions are smooth and easy to learn, others are hard. Left-to-right text suits the way language is structured. For images, every order puts some pixels in a low-context, hard-to-predict spot.
2. **Architecture must match.** The masks in PixelCNN and the causal mask in Transformers encode *which inputs each function may see*. Change the order and the masks must change too.
3. **Errors differ.** With finite capacity and data, f_θ approximates its set of functions imperfectly. Different sets lead to different errors, so two models trained with different orders end up with different learned joints $p_\theta(\mathbf{x})$, even though the true joint factors exactly under both.

In short: the order decides what question the network answers at each step ("given *this*, predict *that*"). A different order is a different question set, so the network is a different function approximator, even when it targets the same joint distribution.

Does a Generative Model Learn Multiple Orders? (Fixed-Order vs. Any-Order Models)

A common misreading is that a generative model learns many different orders of an image so that it can handle any input at inference. This is not how standard AR models work.

A standard AR model learns one fixed order

PixelCNN, PixelRNN and GPT each choose **one order before training**: raster order for images, left-to-right for text. They learn only the conditionals for that order and never see any other.

- **Training:** for every training image, the network learns "given pixels $1, \ldots, i-1$ in raster order, predict pixel i". It learns nothing about reverse order, column order, or anything else.
- **Inference:** it generates in that same raster order, one pixel after another.

The earlier point about different function sets was a comparison *between models*: a separate model trained with a different order would learn a different set of functions. One model does not learn several orders.

Generalizing to new inputs is a separate idea

A standard AR model generalizes because it has seen **many images in the same order**, not because it has seen many orders. When it meets a new context at inference, such as a partly generated face it has never seen, it relies on patterns learned across the training set, for example "skin-colored pixels above usually mean more skin below".

There is one limit. It can only condition on information that comes **earlier in its order**. Inpainting works naturally because known pixels above and to the left act as context. Pixels below or to the right of the hole are ignored by a raster model, even when they are known, because they come later in its order.

Models that do learn many orders

This idea exists as a separate family, **order-agnostic** or **any-order** AR models:

- **Order-agnostic MADE** and **XLNet** sample a random permutation π for each training example, so one network learns conditionals for many orders.
- The benefit is that at inference you can choose the order, for example generating the unknown pixels conditioned on all known pixels wherever they are.
- The cost is a much harder learning problem (many function sets in one network), which usually means worse likelihood than a fixed-order model of the same size.

	Standard AR (PixelCNN, GPT)	Any-order AR (XLNet, MADE)
Orders learned	1, fixed	many, random permutations
Generalizes by	many images in that order	many images $\times$ many orders
Inference order	must follow the trained order	can choose

Directed Graph of an Autoregressive Model

What the graph shows

For four variables, the graph has these arrows:

$$x_1 \to x_2, \quad x_1 \to x_3, \quad x_1 \to x_4, \quad x_2 \to x_3, \quad x_2 \to x_4, \quad x_3 \to x_4.$$

Each **node** is one variable: a pixel, a token, or an audio sample. Each **arrow** $x_j \to x_i$ means "x_i may depend on x_j". The variables that point into a node are its **parents**, and the parents are exactly the context in the chain rule:

Node	Parents	Factor
x_1	none	$p(x_1)$
x_2	x_1	$p(x_2 \mid x_1)$
x_3	x_1, x_2	$p(x_3 \mid x_1, x_2)$
x_4	x_1, x_2, x_3	$p(x_4 \mid x_1, x_2, x_3)$

Every earlier node points to every later node. This is a **fully connected** forward graph, which is why an AR model makes no independence assumptions: the chain rule itself is the graph.

Audio example: with $x_1, \ldots, x_4$ as four consecutive waveform samples, the sample at time 4 may depend on all three previous samples, but sample 2 cannot depend on sample 3.

Why "no cycle" matters

All arrows point forward, so the graph is a **DAG** (directed acyclic graph). That guarantees a valid generation order: start at the node with no parents and move forward. This is ancestral sampling:

1. Sample the first token, "the", from $p(x_1)$.
2. Sample "cat" from $p(x_2 \mid \text{the})$.
3. Continue to the end.

If there were a cycle, say x_2 depends on x_3 and x_3 depends on x_2, neither could be generated first, because each would wait for the other. Sampling would be undefined.

One network with shared weights approximates all conditionals

The graph has D different conditionals, each with a different number of parents. You don't train D separate networks. One network f_θ computes all of them, and masking controls which inputs each output position can see. A Transformer predicting token 4 and token 2 uses the same weights; only the visible context differs.

The causal constraint

Predicting x_i must not depend on $x_i, x_{i+1}, \ldots, x_D$. Graphically, **no arrow may point backward or into a node from itself**. The product is only a valid probability if every factor uses earlier variables alone.

What goes wrong if it leaks: suppose a text model predicting token x_i can see x_i itself. It learns to copy: $p(x_i \mid \ldots, x_i) = 1$ for every token. Then for *any* sentence,

$$\prod_i p(\cdot) = 1 \times 1 \times \cdots = 1.$$

Every possible sentence receives probability 1, and the total over all sentences becomes far greater than 1. It is no longer a distribution, so the likelihood and bpd are meaningless. The model looks perfect during training but generates garbage, because at sampling time the "answer" it learned to copy is not available yet.

Where masking comes in

A CNN or Transformer naturally looks at all positions, which would add backward arrows. Masking deletes those arrows:

- **PixelCNN:** Mask A/B zero out the filter weights that touch the current pixel or future pixels.
- **GPT:** the causal attention mask sets future positions to $-\infty$.

Both enforce exactly this DAG inside the network.

Softmax Output of a Neural Autoregressive Text Model (Slide 8)

$$p(x_t \mid x_{<t}) = \text{softmax}\big(\mathbf{W}\, h_\theta(x_{<t})\big)$$

This equation says how a language model turns "the text so far" into a probability for every possible next token. It happens in three steps.

Step 1: Summarize the context, $h_\theta(x_{<t})$

The network (an RNN or a Transformer) reads all previous tokens $x_1, \ldots, x_{t-1}$ and compresses them into one vector $h \in \mathbb{R}^d$. For example, $d = 768$.

For the context *"The cat sat on the"*, h is a single vector that encodes what this prefix means and what kind of word should come next.

Step 2: Score every vocabulary token, $\mathbf{W}h$

- $\mathbf{W}$ is a $|V| \times d$ matrix, with one row per token in the vocabulary.
- $\mathbf{W}h$ produces a vector $z \in \mathbb{R}^{|V|}$ of **logits**, which are raw, unnormalized scores.
- Each logit is a dot product: $z_k = \mathbf{w}_k^\top h$. You can read it as "how well does token k match this context?"

Logits can be any real number, including negative ones, and they don't sum to anything in particular. So they are not probabilities yet.

Step 3: Convert scores to probabilities (softmax)

$$p(x_t = k \mid x_{<t}) = \frac{\exp(z_k)}{\sum_{j=1}^{|V|} \exp(z_j)}$$

- $\exp(\cdot)$ makes every score positive.
- Dividing by the sum makes all $|V|$ probabilities add up to 1.

Worked example

Context: *"The cat sat on the ___"*. Assume a toy 4-token vocabulary:

Token	Logit z_k	$\exp(z_k)$	Probability
mat	3.0	20.09	0.657
floor	2.0	7.39	0.242
roof	1.0	2.72	0.089
banana	−1.0	0.37	0.012
Sum		30.57	1.000

Note that a gap of 1 in the logits becomes a factor of $e \approx 2.7$ in probability. That is why "mat" dominates even though its score is only slightly higher than "floor".

Why this matters for the lecture

- **Exact likelihood.** Softmax guarantees that each conditional is a *normalized* distribution. Multiplying these conditionals through the chain rule therefore gives a valid p(sentence).
- **Training.** If the true next token is "mat", the loss at this step is $-\log 0.657 \approx 0.42$ nats. Summing this over all positions gives the NLL.
- **Generation.** Sampling draws a token from this distribution, appends it to the context, and repeats (ancestral sampling). Temperature divides z by T *before* the softmax.
- **The "$|V|$ classes" note.** Real vocabularies have tens of thousands of tokens or more, so each generation step is a classification over that many classes. This is the same as PixelCNN's 256-way softmax per color channel, just with a much larger set of classes.

Ordering Image Data for the Chain Rule

Raster Order (Slide 9)

The problem. The chain rule needs a **sequence**, but an image is a 2D grid with 3 color channels. Before you can write $p(\mathbf{x}) = \prod_i p(x_i \mid x_{<i})$, you must decide which value counts as x_1, which as x_2, and so on.

The rule. Raster order flattens the image the way you read a page:

1. Go through rows from top to bottom.
2. Within each row, go through columns from left to right.
3. Within each pixel, go through channels in a fixed order: R, then G, then B.

An $H \times W$ image with C channels becomes a sequence of length

$$D = H \cdot W \cdot C.$$

For example, a 32×32 RGB image gives $D = 32 \cdot 32 \cdot 3 = 3072$ values. The model generates them one at a time.

The 3×3 diagram.

1	2	3
4	5	6
7	8	9

When the model predicts pixel **5**:

- **Context (allowed):** pixels 1–4, i.e. the whole row above plus pixel 4 to its left.
- **Future (hidden):** pixels 6–9.

The slide stresses one point: **pixel 6, the right-hand neighbor, is not context**, even though it is spatially adjacent. Raster order generates it *after* pixel 5, so it doesn't exist yet when pixel 5 is drawn.

Generative-model view. Think of a model painting a face from the top down:

- When it reaches the left eye, it has already drawn the forehead and hairline (rows above). It can use them to decide where the eye goes.
- It **cannot** look at the right eye or the mouth, because those come later in the sequence.
- When it later draws the right eye, it *can* look at the left eye. That is how symmetry gets learned.

Consequence: early pixels (top-left) have almost no context, so the model is uncertain there. Late pixels (bottom-right) have nearly the whole image as context. This is the "order is part of the model" point from Slide 6.

Channel Order Inside One Pixel (Slide 10)

Why not predict R, G, B independently? The three channels of one pixel are **strongly correlated**. Suppose you are generating a sky pixel and predict R, G, B independently from the same context:

- R might sample a sky-like 135.
- G might sample 206.
- B might independently sample 30 instead of $\sim$235.

The result is a muddy olive pixel in the middle of a blue sky. Each channel was plausible on its own, but the combination is wrong.

The fix: chain rule *within* the pixel.

$$p(r, g, b \mid \text{past}) = p(r \mid \text{past})\, p(g \mid r, \text{past})\, p(b \mid r, g, \text{past})$$

This is still the exact chain rule, now applied to the three channels of one pixel:

1. **Red** sees only previous pixels.
2. **Green** sees previous pixels **and** the red value just sampled.
3. **Blue** sees previous pixels, red, **and** green.

In the sky example, once R = 135 and G = 206 are fixed, the model knows the pixel is sky-like, so it concentrates B's probability around 235.

The diagram.

$$x_{<i} \;\to\; R \;\to\; G \;\to\; B$$

Each arrow means "is conditioned on". This is also why, in the full flattened sequence, the order is $R_1\, G_1\, B_1\, R_2\, G_2\, B_2 \ldots$

Output distribution per channel.

- **256-way softmax:** intensities are integers 0–255, so each channel's conditional is a categorical distribution over 256 classes. This is the same softmax as in the text model, with 256 "tokens" instead of a word vocabulary.
- **Mixture of logistics (PixelCNN++):** this treats intensity as roughly continuous. It needs fewer parameters and reflects the fact that 200 and 201 are similar colors, which a softmax doesn't know.

How Raster Order and Channel Order Connect

	Raster order (Slide 9)	Channel order (Slide 10)
Orders	pixels across space	channels within one pixel
Rule	rows top to bottom, columns left to right	R → G → B
Purpose	give the 2D grid a sequence	capture correlation between colors

Together they define the full sequence $x_1, \ldots, x_D$, so the chain rule and exact likelihood apply to images exactly as they do to text.

Exact Likelihood (Slide 11)

Equation 1: Log-Likelihood of One Sample

$$\log p_\theta(\mathbf{x}) = \sum_{i=1}^{D} \log p_\theta(x_i \mid x_{<i})$$

Where it comes from. The chain rule says the joint probability is a **product** of conditionals:

$$p_\theta(\mathbf{x}) = \prod_{i=1}^{D} p_\theta(x_i \mid x_{<i})$$

Take the log of both sides. Since $\log(ab) = \log a + \log b$, the product becomes a **sum**. Nothing is approximated. That is why the slide says the chain rule is an *identity*: the model's likelihood *is* the joint.

Reading each term.

- x_i is the actual value at position i (a token, or one color channel of a pixel).
- $p_\theta(x_i \mid x_{<i})$ is the probability the network assigned to that actual value, given everything before it. It is the softmax output, read off at the correct class.
- log of that probability is always ≤ 0. It is close to 0 when the model was confident and correct, and very negative when the model was surprised.

Text example. Sentence: *"The cat sat"*, with $D = 3$ tokens.

Step	Conditional	Probability	ln
1	$p(\text{The})$	0.05	-2.996
2	$p(\text{cat} \mid \text{The})$	0.10	-2.303
3	$p(\text{sat} \mid \text{The cat})$	0.40	-0.916
	Sum		$\mathbf{-6.215}$

Check: $0.05 \times 0.10 \times 0.40 = 0.002$, and $\ln 0.002 = -6.215$. The two routes agree, as they must.

Why use the log at all? A $32 \times 32 \times 3$ image has 3072 factors. If each one is about 0.1, the product is about 10^{-3072}, which underflows to zero in floating point. The sum of logs is about -7073 nats, which is perfectly representable. The log also turns the problem into a sum of per-position terms, which is easy to compute in parallel and differentiate.

Equation 2: NLL Loss over a Dataset

$$\mathcal{L}(\theta) = -\sum_{n=1}^{N}\sum_{i=1}^{D} \log p_\theta\Big(x_i^{(n)} \mid x_{<i}^{(n)}\Big)$$

Reading it from the inside out.

1. **Inner sum over i:** Equation 1 applied to the n-th training example, which gives its total log-likelihood.
2. **Outer sum over n:** add that up over all N training examples (all images, or all sentences).
3. **Minus sign:** log-likelihood is something we want to *maximize*, but optimizers *minimize*. Negating it gives the **negative log-likelihood (NLL)**. Minimizing NLL is the same as maximizing likelihood.

The superscript (n) just labels which training example a value comes from.

Image example. Suppose you train a PixelCNN on 50,000 CIFAR-10 images, each with $D = 3072$ values:

- Every image contributes 3072 terms, one per color channel of every pixel.
- Each term is the **cross-entropy** of a 256-way softmax against the true intensity.
- $\mathcal{L}$ is the sum of about $50{,}000 \times 3072 \approx 154$ million such terms. In practice you use the minibatch average.

So training an AR model is just **very many classification problems**, all sharing one network. For each position the question is: given the true prefix, did you put high probability on the true next value?

Concrete intuition. If the model generates a sky region and has learned that blue follows blue, it gives the true next pixel high probability. That term is small, for example $-\log 0.8 = 0.22$. If it put probability 0.01 on the true value, the term is $-\log 0.01 = 4.6$, a large penalty. Gradient descent pushes probability mass toward what actually appears in real data.

Why AR Models Give Genuine Maximum Likelihood

The slide's key box reads: *no variational gap, no discriminator, no unknown partition function; if the network outputs a normalized distribution for every conditional, SGD on $\mathcal{L}$ is genuine maximum likelihood.* This makes two claims:

1. Other generative model families have an obstacle that keeps them from computing or optimizing $\log p(\mathbf{x})$ directly.
2. AR models avoid all three obstacles, **provided** every conditional is normalized.

"No variational gap" (vs. VAEs). A VAE generates an image by first sampling a latent $\mathbf{z}$ (pose, style, lighting) and then decoding it. Its likelihood is

$$p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}.$$

This integral covers every possible latent code, so it can't be computed. The VAE instead maximizes a lower bound, the ELBO:

$$\log p(\mathbf{x}) = \text{ELBO} + \underbrace{\text{KL}\big(q(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z} \mid \mathbf{x})\big)}_{\text{variational gap} \geq 0}$$

Improving the ELBO doesn't guarantee you improved the true likelihood, because you might only have shrunk the gap. The number you report is also a bound, not the actual value. An AR model has no $\mathbf{z}$ and no integral: the chain-rule sum *is* $\log p(\mathbf{x})$ exactly.

"No discriminator" (vs. GANs). A GAN generator maps noise to an image, but it never defines $p(\mathbf{x})$ at all. You can't ask "how likely is this image?" Training relies on a second network, the discriminator, that tries to tell real images from fakes, and the two networks play a minimax game. There is no likelihood number to report or compare across models, and training can be unstable (oscillation, or mode collapse such as generating only a few kinds of faces). An AR model has one network and one loss to minimize, with no adversary.

"No unknown partition function" (vs. energy-based models). An energy-based model gives each image a score $E(\mathbf{x})$ and defines

$$p(\mathbf{x}) = \frac{\exp(-E(\mathbf{x}))}{Z}, \qquad Z = \sum_{\text{all possible images}} \exp(-E(\mathbf{x})).$$

For a $32 \times 32 \times 3$ image, Z would be a sum over 256^{3072} images. That is impossible to compute, so you never know the actual probability of any image. In an AR model, normalization happens **locally**, one position at a time. Each softmax sums over only 256 intensities (or $|V|$ tokens), which is cheap. If every local piece sums to 1, the chain rule guarantees the full joint also sums to 1 over all images. The impossible global sum is never needed.

"SGD on $\mathcal{L}$ is genuine maximum likelihood." "Genuine" means you are optimizing the true objective, not a substitute for it:

- $\mathcal{L}$ **equals** the exact negative log-likelihood. It is not a bound (VAE), not a game (GAN), and not an approximation with an unknown Z (EBM).
- A minibatch gradient is an unbiased estimate of the full-dataset gradient of that exact NLL.
- Maximizing likelihood is equivalent to minimizing $\text{KL}(p_{\text{data}} \,\|\, p_\theta)$, which is the definition of a good generative model from Slide 2.

In practice, training a PixelCNN or GPT is just cross-entropy classification at each position. It is stable, it is straightforward to debug, and the final loss is a true likelihood you can convert to bits per dimension and compare across papers.

Summary.

Model	What blocks exact $\log p(\mathbf{x})$	Training objective
VAE	intractable integral over $\mathbf{z}$	lower bound (ELBO)
GAN	no density defined	adversarial game
EBM	intractable global Z	approximate methods
AR	**nothing, if normalized and causal**	**exact NLL**

Meaning of "Every Conditional Is Normalized"

It means that **at every position, for whatever context the model sees, the probabilities it assigns to all possible next values add up to exactly 1.** For continuous values, the density integrates to 1.

Formally, for every position i and every possible prefix $x_{<i}$:

$$\sum_k p_\theta(x_i = k \mid x_{<i}) = 1, \qquad p_\theta(x_i = k \mid x_{<i}) \geq 0.$$

Text example. Context: *"The cat sat on the ___"*

Next token	Probability
mat	0.657
floor	0.242
roof	0.089
banana	0.012
Sum	**1.000**

This must hold for this context, for *"I love ___"*, for the empty context at position 1, and for every other prefix. A softmax guarantees it automatically.

Image example. When predicting the red channel of a pixel, given all pixels above and to the left, the 256 probabilities for intensities $0, 1, \ldots, 255$ must sum to 1. The same holds for green given red, and blue given red and green.

What "not normalized" looks like. Suppose the network outputs raw logits like mat = 3.0, floor = 2.0, roof = 1.0, banana = −1.0. These are scores, not probabilities. They don't sum to 1, and one of them is negative. Multiplying such numbers through the chain rule wouldn't give a real $p(\mathbf{x})$.

Why normalization is essential for training. Imagine a text model that outputs raw scores and is trained to "maximize the score of the true next word." It can cheat by pushing **every** word's score toward infinity. The loss improves, but the model learns nothing. With softmax, probabilities must sum to 1, so the model has a fixed budget: raising $p(\text{"mat"} \mid \text{"The cat sat on the"})$ forces it to lower the probability of "banana." This competition is what forces the model to learn which continuations are actually likely.

What counts as normalized.

Data	Normalized conditional
Text token / pixel intensity (0–255)	softmax over $\lvert V \rvert$ or 256 classes
Binary pixel	Bernoulli via sigmoid
Real-valued audio sample	Gaussian or mixture of logistics (integrates to 1)

Implicit second condition: causality. Each conditional may only see $x_{<i}$. If a mask leaks x_i itself, the model can copy the answer, every factor approaches 1, and the "likelihood" becomes meaningless (Slide 40, Q3).

Why it matters. If each local conditional sums to 1, the chain rule guarantees that the full joint $p(\mathbf{x})$ also sums to 1 over **all** possible sentences or images. That makes it a true probability distribution, so

its log is an exact likelihood. You get global normalization for free from cheap local normalization: a 256-way or $|V|$-way sum at each step, instead of the impossible sum over every image that energy-based models would need.